

Assignment_12.5

Batch_30

2303A510i9

Task Description #1 :(Sorting – Merge Sort Implementation)

Prompt: Generate a Python program that implements the Merge Sort algorithm.

Code &Output:

```
1 #Generate a Python program that implements the Merge Sort algorithm.
2 #instructions: create a function merge_sort(arr) that sorts a list in ascending order,include time comple
3 def merge_sort(arr):
4     """
5     Sorts a list in ascending order using the Merge Sort algorithm.
6
7     Time Complexity:  $O(n \log n)$  - where n is the number of elements in the list.
8     Space Complexity:  $O(n)$  - due to the temporary arrays used during merging.
9
10    Parameters:
11    arr (list): The list of elements to be sorted.
12
13    Returns:
14    list: A new list containing the sorted elements.
15    """
16    if len(arr) <= 1:
17        return arr
18
19    mid = len(arr) // 2
20    left_half = merge_sort(arr[:mid])
21    right_half = merge_sort(arr[mid:])
22
23    return merge(left_half, right_half)
24 def merge(left, right):
25     """
26     Merges two sorted lists into a single sorted list.
27     Parameters:
28     left (list): The first sorted list.
29     right (list): The second sorted list.
30     """
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

TERMINAL

```
C:\Users\harik\OneDrive\Desktop\ai assissted> python assignment12_5.py
Sorted array: [3, 9, 10, 27, 38, 43, 82]
Sorted array: [1, 2, 5, 5, 6, 9]
Sorted array: [5, 6, 7, 11, 12, 13]
C:\Users\harik\OneDrive\Desktop\ai assissted>
```

Task Description #2 : (Searching – Binary Search with AI Optimization)

Prompt: create a binary search function that finds a target element in a sorted list

Code&Output:

```
71 #create a binary search function that finds a target element in a sorted list.
72 #instructions:create a function binary_search(arr,target) returning the index of the target or -1
73 def binary_search(arr, target):
74     """
75     Performs binary search to find the index of the target element in a sorted list.
76
77     Time Complexity:
78     - Best Case: O(1) - when the target is at the middle of the list.
79     - Average Case: O(log n) - where n is the number of elements in the list.
80     - Worst Case: O(log n) - when the target is not present in the list.
81
82     Space Complexity: O(1) - as we are using a constant amount of space.
83
84     Parameters:
85     arr (list): A sorted list of elements.
86     target: The element to search for.
87
88     Returns:
89     int: The index of the target element if found, otherwise -1.
90     """
91     left, right = 0, len(arr) - 1
92
93     while left <= right:
94         mid = left + (right - left) // 2
95
96         # Check if the target is present at mid
97         if arr[mid] == target:
98             return mid
99         # If target is greater, ignore the left half
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

▼ TERMINAL

Target 5 found at index: 4
Target 11 found at index: -1
Target 1 found at index: 0
Target 10 found at index: 9
PS C:\Users\harik\OneDrive\Desktop\ai assisted>

Task Description #3: Smart Healthcare Appointment Scheduling System

Prompt: Design a healthcare appointment system in Python. The system stores appointment ID, patient name, doctor name, appointment time, and consultation fee. Recommend suitable searching and sorting algorithms, explain why they are appropriate, and implement them in Python to:

1. Search appointments by appointment ID.

2. Sort appointments by appointment time or consultation fee.

Code&output:

```
131 class Appointment:
132     def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_f
133         self.appointment_id = appointment_id
134         self.patient_name = patient_name
135         self.doctor_name = doctor_name
136         self.appointment_time = appointment_time
137         self.consultation_fee = consultation_fee
138 class AppointmentSystem:
139     def __init__(self):
140         self.appointments = []
141     def add_appointment(self, appointment):
142         self.appointments.append(appointment)
143     def search_by_id(self, appointment_id):
144         """
145         Searches for an appointment by its ID using linear search.
146         Time Complexity: O(n) - where n is the number of appointments.
147         Space Complexity: O(1) - as we are using a constant amount of space.
148         """
149         for appointment in self.appointments:
150             if appointment.appointment_id == appointment_id:
151                 return appointment
152         return None
153     def sort_by_time(self):
154         """
155         Sorts appointments by appointment time using the built-in sorted function.
156         Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
157         Space Complexity: O(n) - as sorted() creates a new list.
158         """
159         self.appointments = sorted(self.appointments, key=lambda x: x.appointment_time)
160
161 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
162
163 / TERMINAL
164
165 Appointments sorted by consultation fee:
166 John Doe with Dr. Smith at 2023-10-01 10:00 for $100
167 Alice Smith with Dr. Johnson at 2023-10-01 09:30 for $120
168 Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
```

Task Description #4: Railway Ticket Reservation System

Prompt: Design a railway reservation system in Python. The system stores ticket ID, passenger name, train number, seat number, and travel date. Recommend efficient searching and sorting algorithms, explain why they are suitable, and implement them in Python to:

1. Search tickets by ticket ID.
2. Sort bookings by travel date or seat number.

Code&output:

```
191 class Ticket:
192     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
193         self.ticket_id = ticket_id
194         self.passenger_name = passenger_name
195         self.train_number = train_number
196         self.seat_number = seat_number
197         self.travel_date = travel_date
198 class RailwayReservationSystem:
199     def __init__(self):
200         self.tickets = []
201     def add_ticket(self, ticket):
202         self.tickets.append(ticket)
203     def search_by_id(self, ticket_id):
204         """
205         Searches for a ticket by its ID using linear search.
206         Time Complexity: O(n) - where n is the number of tickets.
207         Space Complexity: O(1) - as we are using a constant amount of space.
208         """
209         for ticket in self.tickets:
210             if ticket.ticket_id == ticket_id:
211                 return ticket
212         return None
213     def sort_by_date(self):
214         """
215         Sorts tickets by travel date using the built-in sorted function.
216         Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
217         Space Complexity: O(n) - as sorted() creates a new list.
218         """
219         self.tickets = sorted(self.tickets, key=lambda x: x.travel_date)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

▼ **TERMINAL**

```
Tickets sorted by seat number:
Alice Smith on Train A seat 10C for travel on 2023-10-01
John Doe on Train A seat 12A for travel on 2023-10-01
Jane Doe on Train B seat 15B for travel on 2023-10-02
```

Task Description #5: Smart Hostel Room Allocation System

Prompt: Design a hostel management system in Python. The system stores student ID, room number, floor, and allocation date.

Recommend optimized searching and sorting algorithms, explain why they are suitable, and implement them in Python to:

1. Search allocation details by student ID.
2. Sort records by room number or allocation date.

Code&Output:

```
251 class Student:
252     def __init__(self, student_id, room_number, floor, allocation_date):
253         self.student_id = student_id
254         self.room_number = room_number
255         self.floor = floor
256         self.allocation_date = allocation_date
257 class HostelManagementSystem:
258     def __init__(self):
259         self.students = []
260     def add_student(self, student):
261         self.students.append(student)
262     def search_by_id(self, student_id):
263         """
264         Searches for a student's allocation details by their ID using linear search.
265         Time Complexity: O(n) - where n is the number of students.
266         Space Complexity: O(1) - as we are using a constant amount of space.
267         """
268         for student in self.students:
269             if student.student_id == student_id:
270                 return student
271         return None
272     def sort_by_room_number(self):
273         """
274         Sorts student records by room number using the built-in sorted function.
275         Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
276         Space Complexity: O(n) - as sorted() creates a new list.
277         """
278         self.students = sorted(self.students, key=lambda x: x.room_number)
279     def sort_by_allocation_date(self):
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

```
TERMINAL

Records sorted by allocation date:
ID 1 allocated to room 101A on floor 1 since 2023-10-01
ID 3 allocated to room 201C on floor 2 since 2023-10-01
ID 2 allocated to room 102B on floor 1 since 2023-10-02
PS C:\Users\harik\OneDrive\Desktop\ai assisted>
```

Task Description #6: Online Movie Streaming Platform

Prompt: Design a streaming service system in Python that stores movie ID, title, genre, rating, and release year, and recommend efficient searching and sorting algorithms, justify why they are suitable, and implement Python functions to search movies by movie ID and sort movies based on rating or release year.

Code & Output:

```

310 class Movie:
311     def __init__(self, movie_id, title, genre, rating, release_year):
312         self.movie_id = movie_id
313         self.title = title
314         self.genre = genre
315         self.rating = rating
316         self.release_year = release_year
317 class StreamingService:
318     def __init__(self):
319         self.movies = []
320     def add_movie(self, movie):
321         self.movies.append(movie)
322     def search_by_id(self, movie_id):
323         """
324         Searches for a movie by its ID using linear search.
325         Time Complexity: O(n) - where n is the number of movies.
326         Space Complexity: O(1) - as we are using a constant amount of space.
327         """
328         for movie in self.movies:
329             if movie.movie_id == movie_id:
330                 return movie
331         return None
332     def sort_by_rating(self):
333         """
334         Sorts movies by rating using the built-in sorted function.
335         Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
336         Space Complexity: O(n) - as sorted() creates a new list.
337         """

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

TERMINAL

```

Movies sorted by release year:
The Matrix (Action) with rating 8.7 released in 1999
Inception (Sci-Fi) with rating 8.8 released in 2010
Interstellar (Sci-Fi) with rating 8.6 released in 2014

```

Task Description #7: Smart Agriculture Crop Monitoring System

Prompt: Design an agriculture monitoring system in Python that stores crop ID, crop name, soil moisture level, temperature, and yield estimate, and use AI-assisted reasoning to recommend optimized searching and sorting algorithms, justify why they are suitable, and implement Python functions to search crop details by crop ID and sort crops based on moisture level or yield estimate.

Code& output:

```

370 class Crop:
371     def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
372         self.crop_id = crop_id
373         self.crop_name = crop_name
374         self.soil_moisture_level = soil_moisture_level
375         self.temperature = temperature
376         self.yield_estimate = yield_estimate
377 class AgricultureMonitoringSystem:
378     def __init__(self):
379         self.crops = []
380     def add_crop(self, crop):
381         self.crops.append(crop)
382     def search_by_id(self, crop_id):
383         """
384         Searches for crop details by crop ID using linear search.
385         Time Complexity: O(n) - where n is the number of crops.
386         Space Complexity: O(1) - as we are using a constant amount of space.
387         """
388         for crop in self.crops:
389             if crop.crop_id == crop_id:
390                 return crop
391         return None
392     def sort_by_moisture_level(self):
393         """
394         Sorts crops by soil moisture level using the built-in sorted function.
395         Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
396         Space Complexity: O(n) - as sorted() creates a new list.
397         """
398         self.crops = sorted(self.crops, key=lambda x: x.soil_moisture_level)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

TERMINAL

```

Crops sorted by yield estimate:
Corn with soil moisture level 40, temperature 28, and yield estimate 1500
Rice with soil moisture level 35, temperature 22, and yield estimate 1200
Wheat with soil moisture level 30, temperature 25, and yield estimate 1000

```

Task Description #8: Airport Flight Management System

Prompt: Design an airport flight management system in Python that stores flight ID, airline name, departure time, arrival time, and status, and use AI to recommend efficient searching and sorting algorithms, justify why they are suitable, and implement Python functions to search flight details by flight ID and sort flights based on departure time or arrival time.

Code& Output:

```

430 class Flight:
431     def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
432         self.flight_id = flight_id
433         self.airline_name = airline_name
434         self.departure_time = departure_time
435         self.arrival_time = arrival_time
436         self.status = status
437 class AirportFlightManagementSystem:
438     def __init__(self):
439         self.flights = []
440     def add_flight(self, flight):
441         self.flights.append(flight)
442     def search_by_id(self, flight_id):
443         """
444         Searches for flight details by flight ID using linear search.
445         Time Complexity: O(n) - where n is the number of flights.
446         Space Complexity: O(1) - as we are using a constant amount of space.
447         """
448         for flight in self.flights:
449             if flight.flight_id == flight_id:
450                 return flight
451         return None
452     def sort_by_departure_time(self):
453         """
454         Sorts flights by departure time using the built-in sorted function.
455         Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
456         Space Complexity: O(n) - as sorted() creates a new list.
457         """
458         self.flights = sorted(self.flights, key=lambda x: x.departure_time)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

▼ TERMINAL

Flights sorted by arrival time:

Airline C departing at 2023-10-01 07:30 arriving at 2023-10-01 09:30 with status On Time
 Airline A departing at 2023-10-01 08:00 arriving at 2023-10-01 10:00 with status On Time
 Airline B departing at 2023-10-01 09:00 arriving at 2023-10-01 11:00 with status Delayed